CAMPUSMART

NAME:ANSHU SINGH

ENROLLMENT NO:24162121002

BATCH:41

CSE(BDA)

TECH:MERN STACK

- Introduction

  CampusMart is a full-stack web application that allows users to browse, search, and manage products with an interactive user interface.

- Tech Stack
  - React.js (Frontend – main focus)
  - Node.js & Express (Backend – basic mention)
  - MongoDB Atlas (Database)

- FRONTEND CODE

  1. App.js (Routing)

Handles navigation between different pages using React Router.

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import ProductForm from './pages/ProductForm';
import Sell from './pages/Sell';
import ProductDetails from './pages/ProductDetails';
import Wishlist from './pages/Wishlist';
import Cart from './pages/Cart';

function App() {
 const [isAuthenticated, setIsAuthenticated] = React.useState(false);
 const [wishlist, setWishlist] = React.useState([]);

 React.useEffect(() => {
  const storedAuth = localStorage.getItem('campusmart_auth') === 'true';
  setIsAuthenticated(storedAuth);
  const storedWishlist = JSON.parse(localStorage.getItem('campusmart_wishlist') ||
'[]');
  setWishlist(storedWishlist);
 }, []);

 const handleAuth = (value) => {
  setIsAuthenticated(value);
  localStorage.setItem('campusmart_auth', value ? 'true' : 'false');
 };

 const addToWishlist = (product) => {
  setWishlist((prev) => {
   const exists = prev.find((p) => p.id === product.id);
   if (exists) return prev;
   const updated = [...prev, product];
   localStorage.setItem('campusmart_wishlist', JSON.stringify(updated));
   return updated;
  });
 };

 const removeFromWishlist = (productId) => {
```

```
    setWishlist((prev) => {
      const updated = prev.filter((p) => p.id !== productId);
      localStorage.setItem('campusmart_wishlist', JSON.stringify(updated));
      return updated;
    });
  };

  return (
    <Router>
      <Navbar
        isAuthenticated={isAuthenticated}
        onSignOut={() => handleAuth(false)}
        wishlistCount={wishlist.length}
      />
      <div className="app-container">
        <Routes>
          <Route path="/" element={
            <Home
              wishlist={wishlist}
              onAddToWishlist={addToWishlist}
              onRemoveFromWishlist={removeFromWishlist}
            />
          } />
          <Route path="/wishlist" element={
            <Wishlist wishlist={wishlist} onRemove={removeFromWishlist} />
          } />
          <Route path="/post-item" element={<ProductForm />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/login" element={<Login setAuth={handleAuth} />} />
          <Route path="/register" element={<Register setAuth={handleAuth} />} />
          <Route path="/sell" element={<Sell />} />
          <Route path="/add-product" element={<ProductForm />} />
          <Route path="/product/:id" element={<ProductDetails />} />
          <Route path="/cart" element={<Cart />} />
        </Routes>
      </div>
      <Footer />
    </Router>
  );
}

export default App;
```

2. Login.js / Signup.js

Manages user authentication through login and registration forms.

```
import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';

const Login = () => {
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleLogin = (e) => {
    e.preventDefault();
    setError('');

    // localStorage se registered users lo
    const users = JSON.parse(localStorage.getItem('campusUsers') || '[]');
    const found = users.find(u => u.email === email && u.password === password);

    if (found) {
      localStorage.setItem('campusmart_auth', 'true');
      localStorage.setItem('campusUser', JSON.stringify(found));
      navigate('/');
    } else {
      setError('❌ Email ya password galat hai. Pehle Register karo!');
    }
  };

  return (
    <main className="page-shell" style={{ display: 'flex', justifyContent: 'center', alignItems:
'center', minHeight: '80vh' }}>
      <div style={{ background: 'white', borderRadius: '24px', padding: '40px', width: '100%',
maxWidth: '400px', boxShadow: '0 20px 60px rgba(15,23,42,0.12)' }}>
        <h2 style={{ fontSize: '28px', fontWeight: '800', color: '#0f172a', marginBottom:
'6px' }}>Welcome Back 👋</h2>
        <p style={{ color: '#64748b', marginBottom: '28px' }}>Login to your CampusMart
account</p>
```

```
      {error && (
        <div style={{ background: '#fef2f2', border: '1px solid #fecaca', color: '#dc2626',
padding: '12px 16px', borderRadius: '10px', marginBottom: '20px', fontSize: '14px' }}>
          ✕ {error}
        </div>
      )}

      <form onSubmit={handleLogin}>
        <div style={{ marginBottom: '16px' }}>
          <label style={{ fontSize: '14px', fontWeight: '600', color: '#0f172a', display: 'block',
marginBottom: '6px' }}>Email</label>
          <input
            type="email"
            value={email}
            onChange={e => setEmail(e.target.value)}
            placeholder="tumhara@email.com"
            required
            style={{ width: '100%', padding: '12px 16px', border: '2px solid #e2e8f0',
borderRadius: '10px', fontSize: '15px', outline: 'none', boxSizing: 'border-box' }}
          />
        </div>

        <div style={{ marginBottom: '24px' }}>
          <label style={{ fontSize: '14px', fontWeight: '600', color: '#0f172a', display: 'block',
marginBottom: '6px' }}>Password</label>
          <input
            type="password"
            value={password}
            onChange={e => setPassword(e.target.value)}
            placeholder="••••••••"
            required
            style={{ width: '100%', padding: '12px 16px', border: '2px solid #e2e8f0',
borderRadius: '10px', fontSize: '15px', outline: 'none', boxSizing: 'border-box' }}
          />
        </div>

        <button type="submit" style={{
          width: '100%', padding: '14px',
          background: 'linear-gradient(135deg, #ec4899, #f43f5e)',
          color: 'white', border: 'none', borderRadius: '12px',
          fontSize: '16px', fontWeight: '800', cursor: 'pointer',
          boxShadow: '0 6px 20px rgba(236,72,153,0.4)'
        }}>🔑 Login</button>
      </form>

      <p style={{ textAlign: 'center', marginTop: '20px', fontSize: '14px', color: '#64748b' }}>
        Don't have an account?{' '}
        <Link to="/register" style={{ color: '#ec4899', fontWeight: '700', textDecoration:
'none' }}>Register here</Link>
      </p>
```

```
      </div>
    </main>
  );
};

export default Login;
```

3. productService.js

This file handles API calls between frontend and backend.

```
/**
 * Product API Integration Examples
 * Usage examples for frontend integration with the product API
 */

import axios from 'axios';

export const addProduct = async (product) => {
  const res = await fetch("http://localhost:5001/api/products/add", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(product)
  });

  return res.json();
};

const API_BASE_URL = 'http://localhost:5001/api/products';

// Configure axios instance
const productAPI = axios.create({
  baseURL: API_BASE_URL,
  timeout: 10000, // Increased timeout
  headers: {
    'Content-Type': 'application/json'
  }
});

// Add response interceptor for better error handling
productAPI.interceptors.response.use(
  (response) => response,
  (error) => {
```

```
    if (error.code === 'ECONNABORTED') {
      throw new Error('Request timeout. Please check your connection and try again.');
    }
    if (!error.response) {
      throw new Error('Network error. Please check your internet connection and ensure the
server is running.');
    }
    // Re-throw the error with the response data
    throw error;
  }
);

// ============================================
// 1. GET ALL PRODUCTS
// ============================================
export const getAllProducts = async (filters = {}) => {
  try {
    const { category, minPrice, maxPrice, search, sort } = filters;

    const params = new URLSearchParams();
    if (category) params.append('category', category);
    if (minPrice) params.append('minPrice', minPrice);
    if (maxPrice) params.append('maxPrice', maxPrice);
    if (search) params.append('search', search);
    if (sort) params.append('sort', sort);

    const response = await productAPI.get(`/?${params}`);
    return response.data;
  } catch (error) {
    console.error('Error fetching products:', error);
    throw error;
  }
};

// ============================================
// 2. ADD NEW PRODUCT
// ============================================
export const addProduct = async (productData) => {
  try {
    const response = await productAPI.post('/', {
      title: productData.title,
      price: parseFloat(productData.price),
      description: productData.description,
      imageUrl: productData.imageUrl,
      category: productData.category || 'Other',
      stock: productData.stock || 0
    });
    return response.data;
  } catch (error) {
    console.error('Error adding product:', error);
    throw error;
```

```
  }
};

// ============================================
// 3. GET PRODUCT BY ID
// ============================================
export const getProductById = async (productId) => {
  try {
    const response = await productAPI.get(`/${productId}`);
    return response.data;
  } catch (error) {
    console.error(`Error fetching product ${productId}:`, error);
    throw error;
  }
};

// ============================================
// 4. UPDATE PRODUCT
// ============================================
export const updateProduct = async (productId, updateData) => {
  try {
    const response = await productAPI.put(`/${productId}`, updateData);
    return response.data;
  } catch (error) {
    console.error(`Error updating product ${productId}:`, error);
    throw error;
  }
};

// ============================================
// 5. DELETE PRODUCT
// ============================================
export const deleteProduct = async (productId) => {
  try {
    const response = await productAPI.delete(`/${productId}`);
    return response.data;
  } catch (error) {
    console.error(`Error deleting product ${productId}:`, error);
    throw error;
  }
};

// ============================================
// 6. SEARCH PRODUCTS
// ============================================
export const searchProducts = async (query) => {
  try {
    const response = await productAPI.get(`/search?query=${encodeURIComponent(query)}`);
    return response.data;
  } catch (error) {
    console.error('Error searching products:', error);
```

```
    throw error;
  }
};

// ============================================
// 7. GET PRODUCTS BY CATEGORY
// ============================================
export const getProductsByCategory = async (category) => {
  try {
    const response = await productAPI.get(`/category/${category}`);
    return response.data;
  } catch (error) {
    console.error(`Error fetching ${category} products:`, error);
    throw error;
  }
};

// ============================================
// USAGE EXAMPLES
// ============================================

/**
 * Example 1: Fetch all products on component mount
 *
 * useEffect(() => {
 *   const fetchProducts = async () => {
 *     try {
 *       const data = await getAllProducts();
 *       setProducts(data.products);
 *     } catch (error) {
 *       setError('Failed to fetch products');
 *     }
 *   };
 *   fetchProducts();
 * }, []);
 */

/**
 * Example 2: Fetch products with filters
 *
 * const handleFilter = async (category, minPrice, maxPrice) => {
 *   try {
 *     const data = await getAllProducts({
 *       category,
 *       minPrice,
 *       maxPrice,
 *       sort: 'price-asc'
 *     });
 *     setProducts(data.products);
 *   } catch (error) {
 *     console.error('Filter failed');
```

```
 *   }
 * };
 */

/**
 * Example 3: Add new product
 *
 * const handleAddProduct = async (formData) => {
 *   try {
 *     const result = await addProduct({
 *       title: formData.title,
 *       price: formData.price,
 *       description: formData.description,
 *       imageUrl: formData.imageUrl,
 *       category: formData.category,
 *       stock: formData.stock
 *     });
 *     if (result.success) {
 *       setProducts([...products, result.product]);
 *       alert('Product added successfully!');
 *     }
 *   } catch (error) {
 *     alert('Failed to add product');
 *   }
 * };
 */

/**
 * Example 4: Search products
 *
 * const handleSearch = async (searchQuery) => {
 *   try {
 *     const data = await searchProducts(searchQuery);
 *     setProducts(data.products);
 *   } catch (error) {
 *     console.error('Search failed');
 *   }
 * };
 */

/**
 * Example 5: Get products by category
 *
 * const handleCategoryClick = async (category) => {
 *   try {
 *     const data = await getProductsByCategory(category);
 *     setProducts(data.products);
 *   } catch (error) {
 *     console.error('Category fetch failed');
 *   }
 * };
```

```
 */

export default {
  getAllProducts,
  addProduct,
  getProductById,
  updateProduct,
  deleteProduct,
  searchProducts,
  getProductsByCategory
};
```

- BACKEND

  Backend handles API requests and connects the application to MongoDB database.

1. Db.js

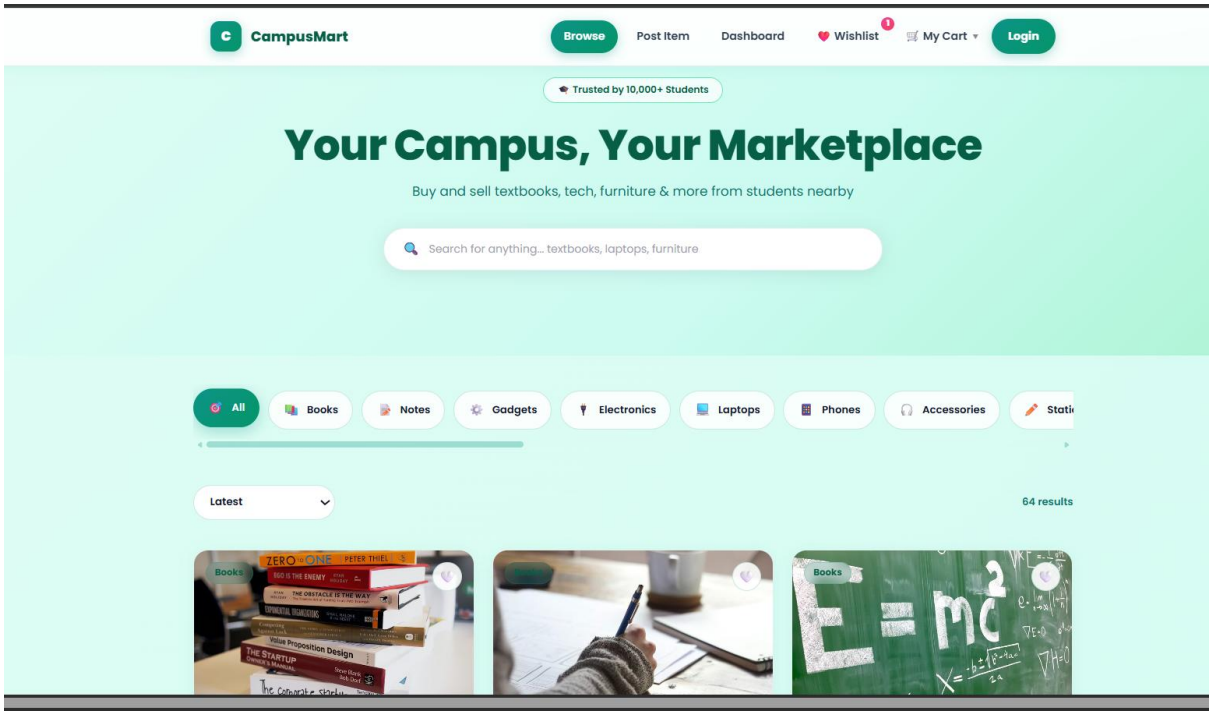
```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 30000,
      socketTimeoutMS: 45000,
    });
    console.log(`☑ MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`❌ DB Error: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;
```

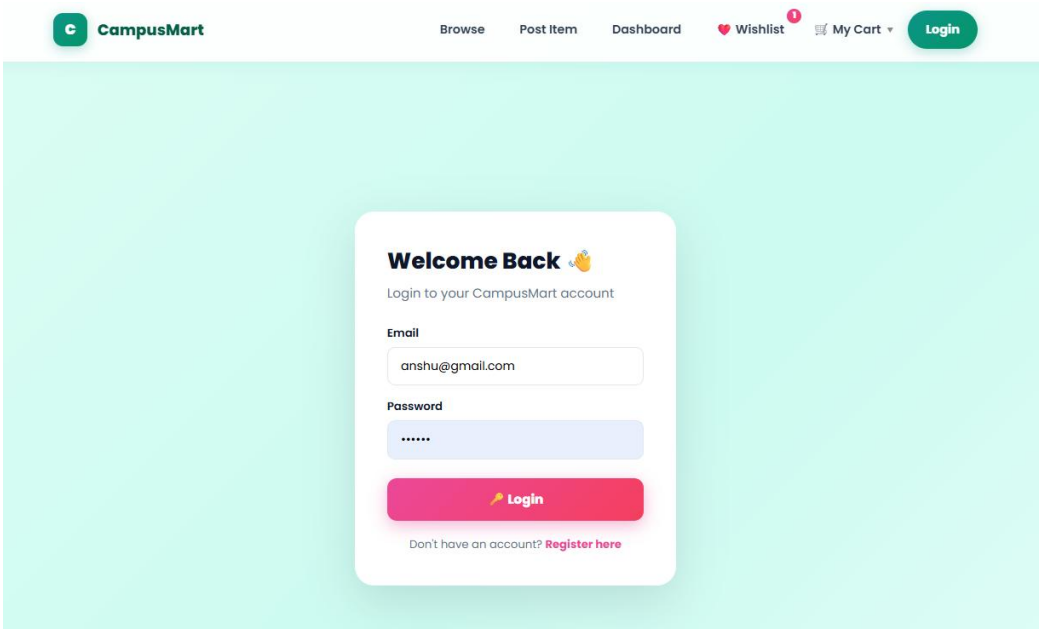
2. productRoute.js

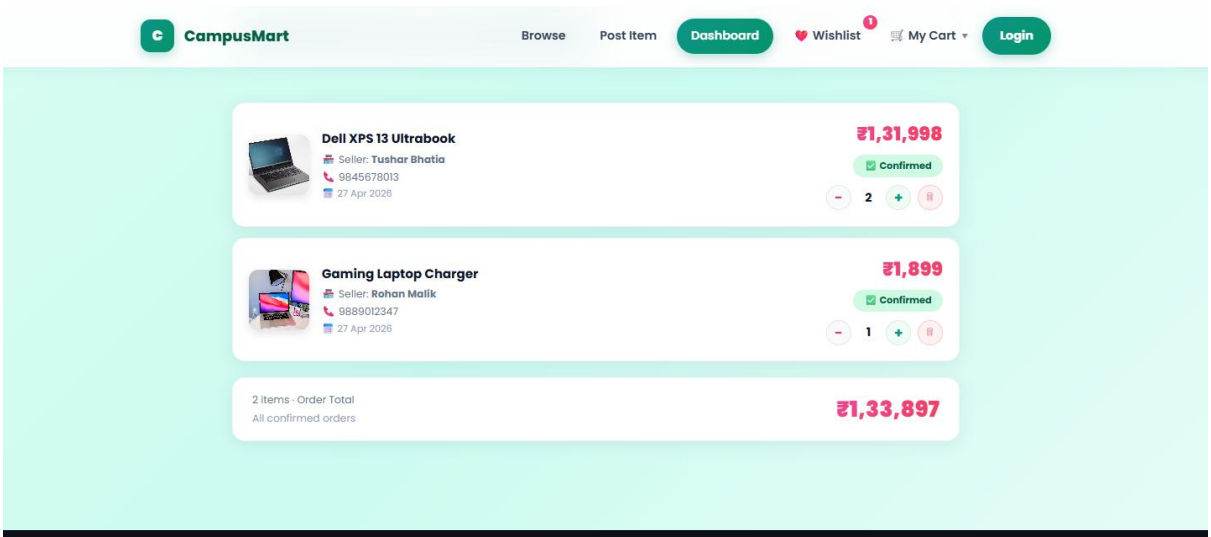
```
const express = require("express");
const router = express.Router();
const Product = require("../models/Product");
```

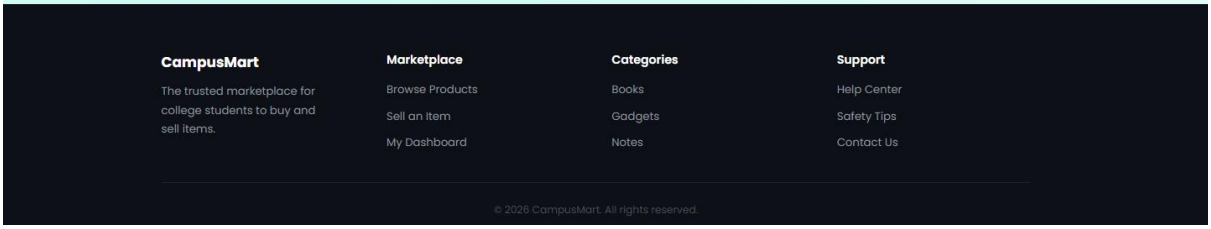
```
// SAVE DATA TO MONGODB
router.post("/add", async (req, res) => {
  const product = new Product(req.body);
  const saved = await product.save();
  res.json(saved);
});

module.exports = router;
```

Output Screenshots:




CampusMart
Browse
Post Item
Dashboard
Wishlist
My Cart
Login
Welcome Back
Login to your CampusMart account
Email
anshu@gmail.com
Password
Login
Don't have an account? Register here


CampusMart
Browse
Post Item
Dashboard
Wishlist
My Cart
Login
Dell XPS 13 Ultrabook
Seller: Tushar Bhatia
9845678013
27 Apr 2026
₹1,31,998
Confirmed
2
Gaming Laptop Charger
Seller: Rohan Malik
9889012347
27 Apr 2026
₹1,899
Confirmed
1
2 items · Order Total
All confirmed orders
₹1,33,897


CampusMart
The trusted marketplace for college students to buy and sell items.
Marketplace
Browse Products
Sell an Item
My Dashboard
Categories
Books
Gadgets
Notes
Support
Help Center
Safety Tips
Contact Us
© 2026 CampusMart. All rights reserved.

Post Item

List your item

Upload details and share your listing with the campus community.

Product Image

Drag & drop an image

or click to browse your files

Title * e.g. Calculus Book

Price (₹) * e.g. 250

Category Books

Condition

New | Like New | Good | Fair

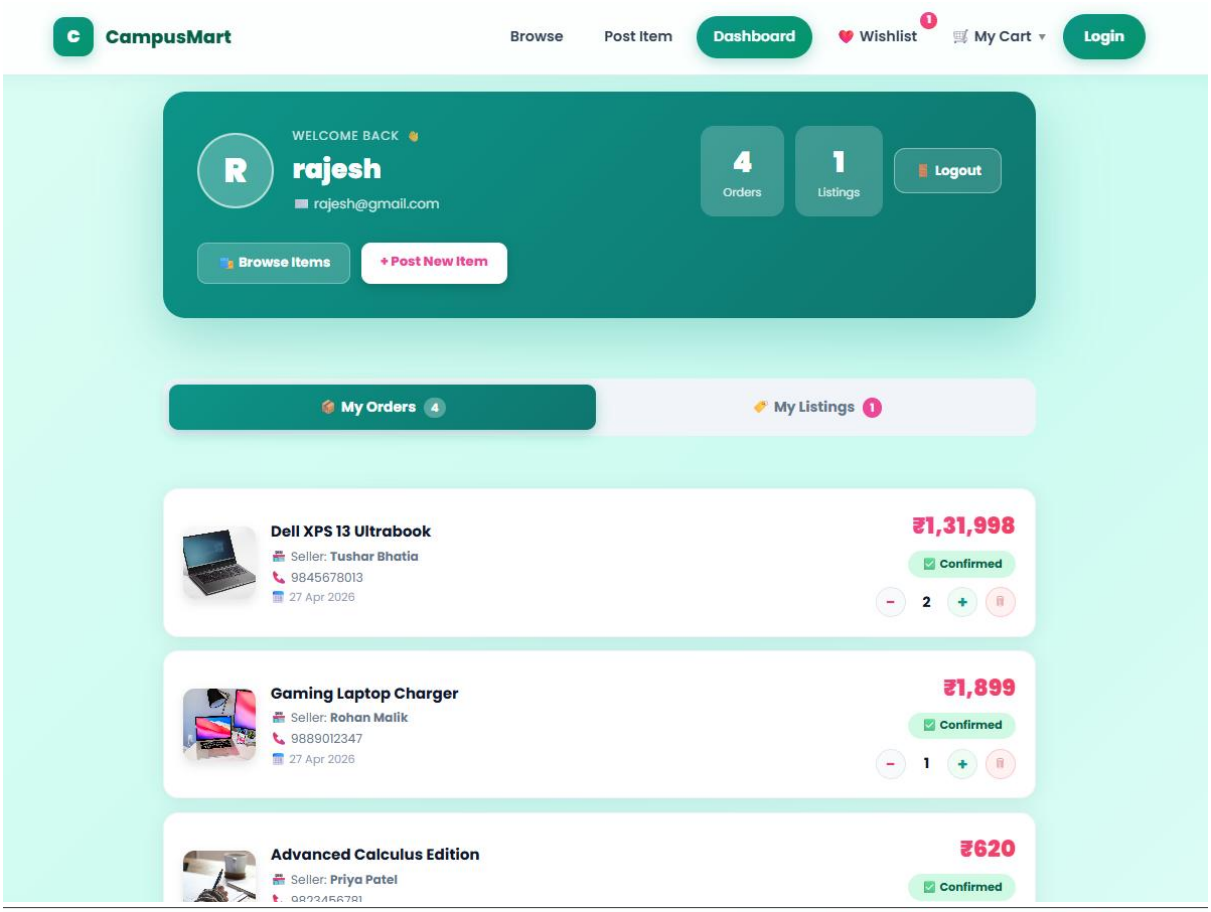
Description *

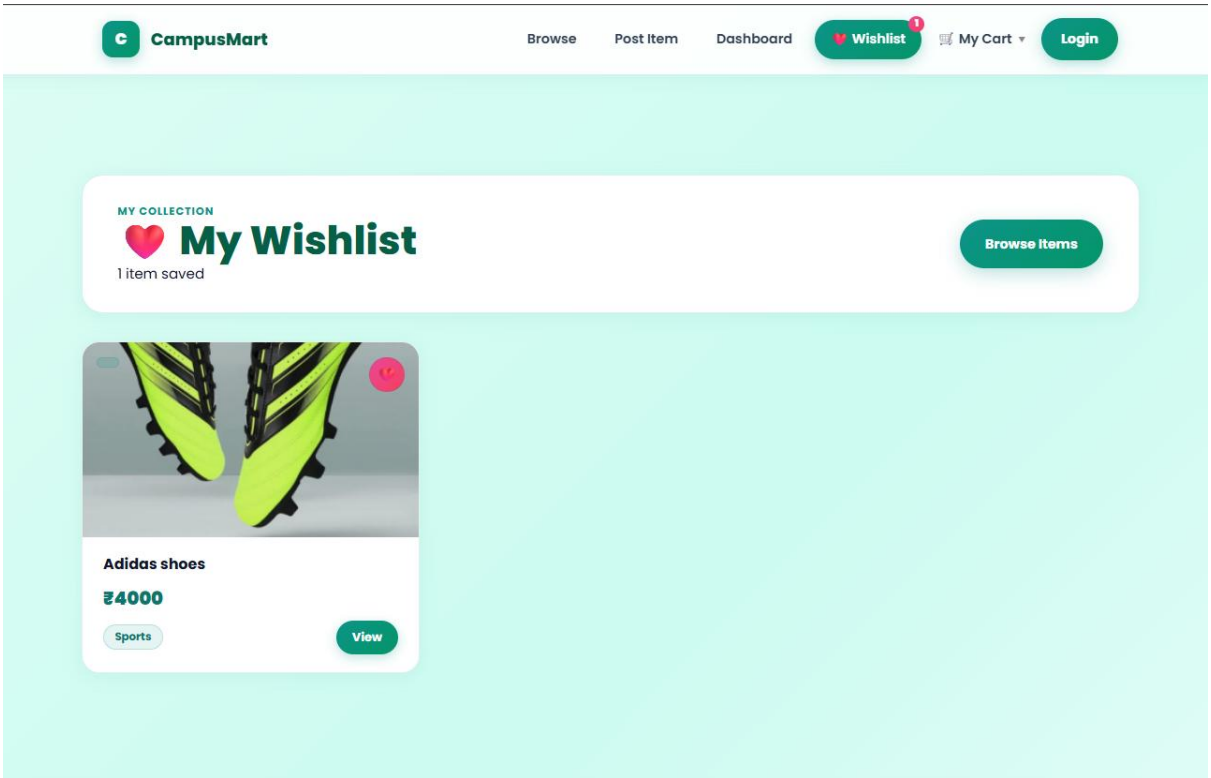
Describe your item — condition, usage, what's included...

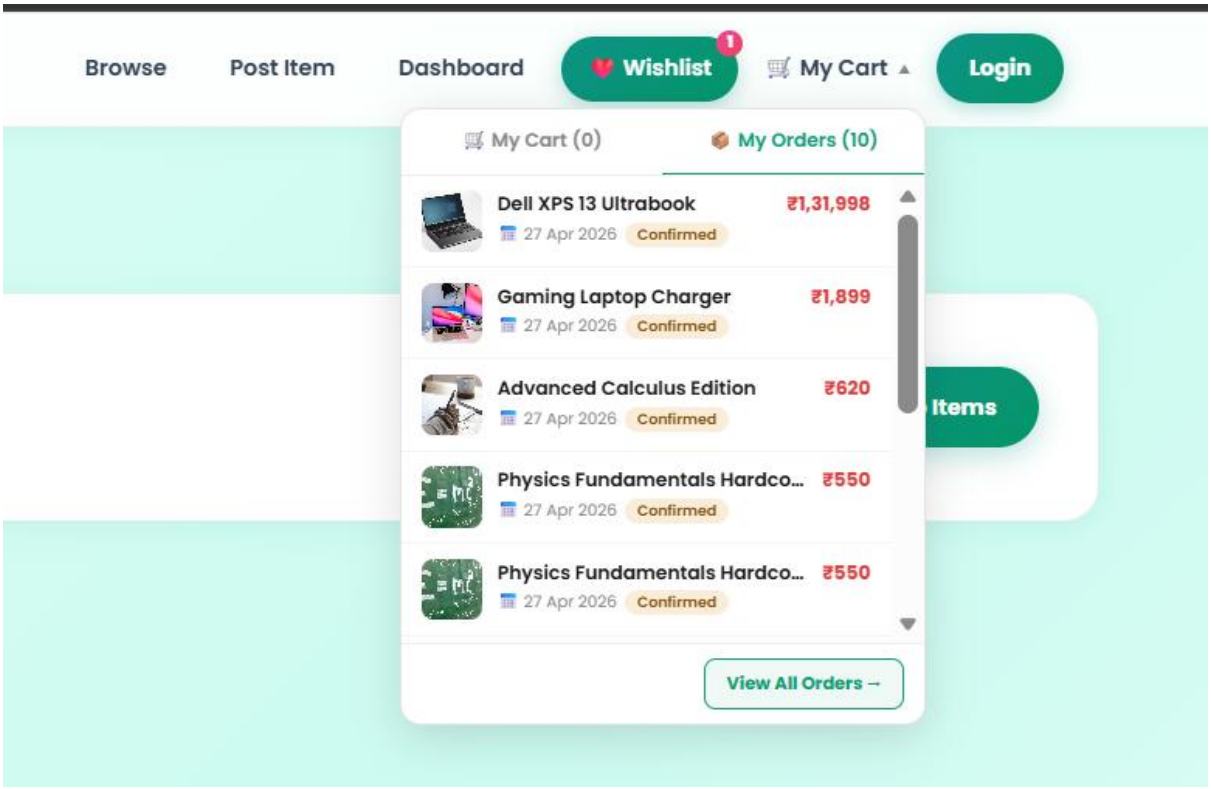
Contact Number 98XXXXXXXX

Location e.g. Hostel A, Room 101

Publish Listing | Cancel


CampusMart
Browse
Post Item
Dashboard
Wishlist
1
My Cart
Login
WELCOME BACK
R
rajesh
rajesh@gmail.com
4
Orders
1
Listings
Logout
Browse Items
+ Post New Item
My Orders 4
My Listings 1
Dell XPS 13 Ultrabook
Seller: Tushar Bhatia
9845678013
27 Apr 2026
₹1,31,998
Confirmed
2
Gaming Laptop Charger
Seller: Rohan Malik
9889012347
27 Apr 2026
₹1,899
Confirmed
1
Advanced Calculus Edition
Seller: Priya Patel
₹620
Confirmed


CampusMart
Browse
Post Item
Dashboard
Wishlist
1
My Cart
Login
MY COLLECTION
My Wishlist
1 item saved
Browse Items
Adidas shoes
₹4000
Sports
View



Conclusion:

This project demonstrates a responsive frontend with dynamic data handling using React and API integration.